
IA301 - Report on Ontologies

An Ontology for University Structure and Relationships

Liam Loughman
Institut Polytechnique de Paris
liam.loughman@ip-paris.fr

Mathieu Antonopoulos
Institut Polytechnique de Paris
Antonopoulos@ip-paris.fr

Baptiste Geisenberger
Institut Polytechnique de Paris
geisenberger@ip-paris.fr

Pablo Bertaudo-Velten
Institut Polytechnique de Paris
pablo.bertaudoelten@gmail.com

Tim Luka Horstmann
Institut Polytechnique de Paris
tim.horstmann@ip-paris.fr

Abstract

This report presents an ontology designed to model the structure, relationships, and roles within a university system. We developed the ontology using Stanford's Protégé tool. After creating the ontology, we evaluated its capabilities through selected SPARQL queries. Potential future improvements include expanding data properties and enhancing the ontology's granularity for broader application.

Contents

1	Introduction	2
2	Ontology Design	2
2.1	Classes	2
2.2	Data Properties	4
2.3	Object Properties	4
2.4	Individuals	5
3	Reasoning and Inferences	7
4	Example Queries	9
4.1	Queries Overview	9
4.2	Query Examples	9
5	Conclusion	13

1 Introduction

The task for this practical work in the IA301 class was to design an ontology from scratch and implement it in Protégé. Given the open-ended nature and flexibility of this assignment, we chose a modeling problem with practical relevance: developing an ontology to represent a university's structure, relationships, and roles. We specifically selected this topic as it allows for exploring the complex hierarchical and relational structures of large organisations, which can be generalized to various domains beyond academia.

The following sections detail the ontology design (Section 2), the types of reasoning and inferences (Section 3), as well as SPARQL queries (Section 4) that can be executed against the ontology in Protégé.

All work described in this report was contributed equally by the five authors listed above.

2 Ontology Design

This section details the design of the university ontology created for this practical exercise. In particular, the *classes*, *data properties*, *object properties*, and *individuals* are explained.

2.1 Classes

We identified key classes that represent the core components of the university system. These classes include:

- **University**: Represents the overall institution.
- **Program**: Represents different levels of academic programs. The available academic programs are subdivided into **Undergraduate** (includes **Bachelor**) and **Postgraduate** (includes **PhD** and **Master (M1 & M2)**) programs.
- **Person**: A general class to represent individuals. For the given task, we divided this class further into the subclasses **Employee** and **Student**. The former is further split into subclasses, representing the different types of employees usually found at a university: **Cleaner**, **Cook**, **Professor**, **Researcher**, **Secretary**.
- **Period**: Period/term of an academic year.
- **Department**: Represents different departments within the university.
- **Major**: Represents the Major the students can choose.
- **Course**: Represents individual courses.
- **Country**: Country the university is located in.
- **City**: City the university is located in.
- **Building**: Represents the physical structure

The hierarchical structure of these classes is visualized in Figure 1, below.

A screenshot of the class hierarchy taken from Protégé is depicted in Figure 2.

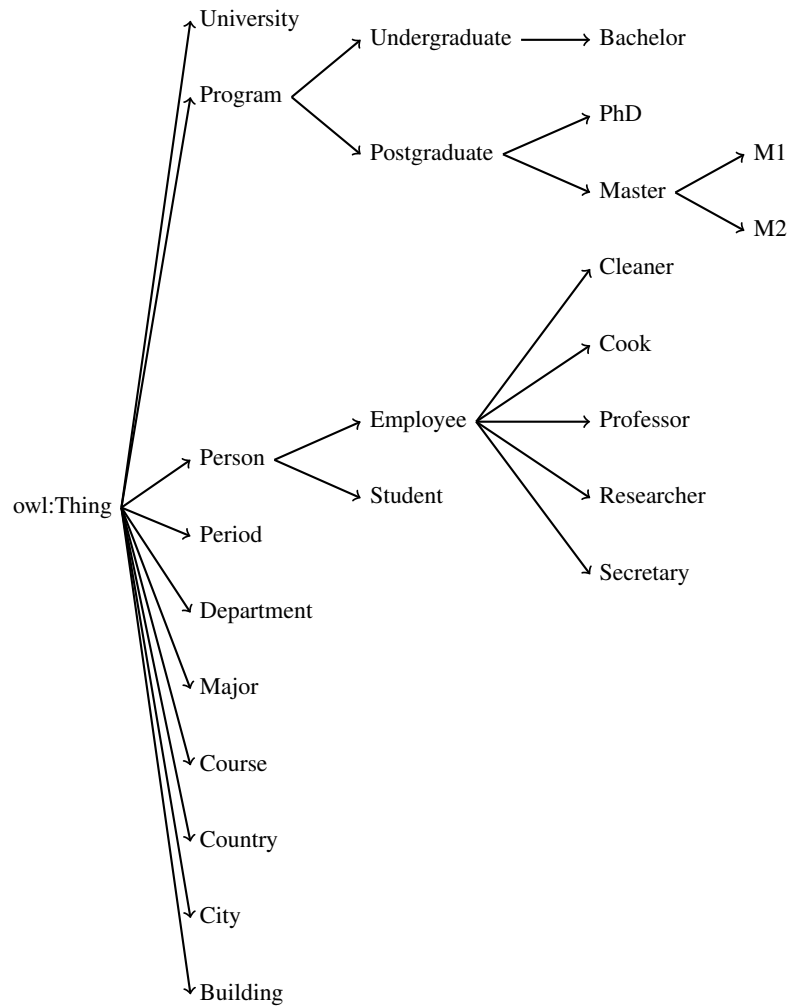


Figure 1: Class Hierarchy in the University Ontology

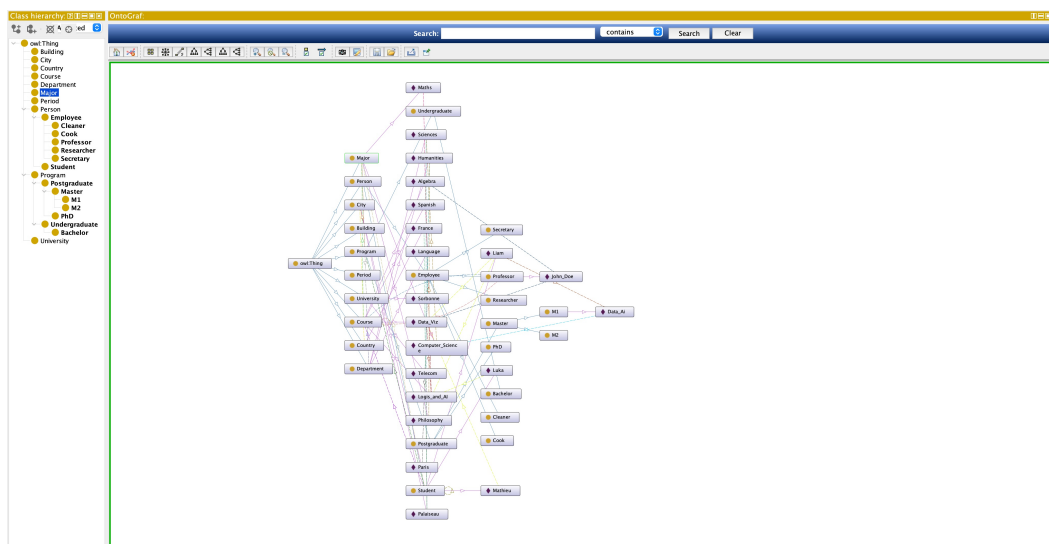


Figure 2: Class Hierarchy in the University Ontology (Screenshot from Protégé)

2.2 Data Properties

We defined the following data properties:

- **Gender**: A string representing the gender of a person.
- **GPA**: A decimal number representing the GPA of a student.
- **Salary**: A decimal number representing the yearly salary of an employee.
- **Credits**: A decimal number representing the amount of credits that is worth a course.

2.3 Object Properties

We defined several object properties to capture relationships between the entities defined in Section 2.1:

- **isTaughtBy**: Links a course to the professors teaching it.
- **Teaches**: This property, inverse of the previous one, links a professor to the course they teach.
- **Enrolled_In**: Associates students to programs they are enrolled.
- **Located_In**: This property indicates the location of departments and buildings within the university. This property is also transitive.
- **Belongs_To**: Associates a course to a major.
- **HasPrerequisite**: This property specifies prerequisites for a course, indicating other courses that must be completed before enrolment.
- **isCityOf**: This property associates a city to a country. It helps to locate the university on a different geographical scale.
- **isClassmate_of**: This property associates two students enrolled in the same course. It is a symmetric property. If A is classmate of B, B has to be classmate of A. It also has the following super-property: **Enrolled_In o inverse (Enrolled_In)** that automatically associates classmates based on their courses.
- **Majors_In**: Associates a student to a program.
- **PartOf**: This property associates a program to a university.
- **Teaches**: This property associates a professor with the course they teach.

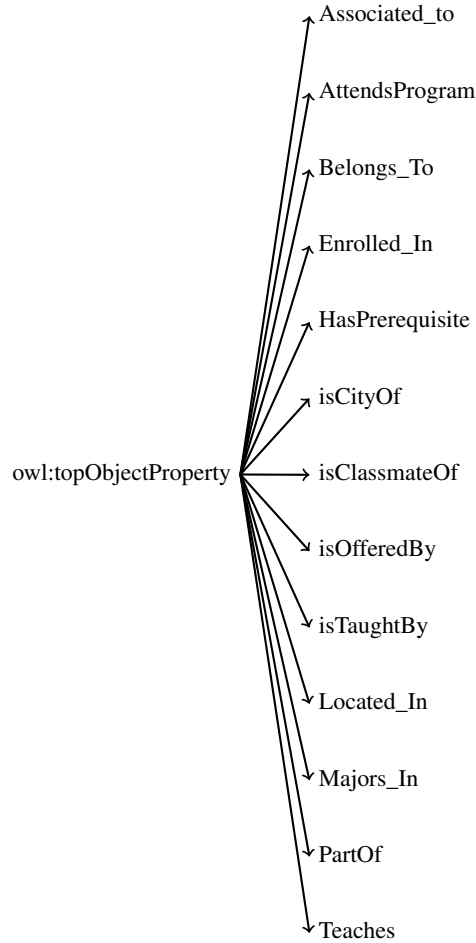


Figure 3: Object Property Hierarchy in the University Ontology

2.4 Individuals

We defined the following different individuals in this ontology:

- **Algebra, Data_Ai, Spanish, Data_Viz, Logics_and_AI:** These are instances of the course class, representing specific courses offered in different programs.
- **Liam:** An instance of the student class. Liam is enrolled in two courses: *Logics_and_AI* and *Data_Viz*, as represented by the *Enrolled_In* property. Therefore, Liam has several classmates inferred by the reasoner (*Mathieu* and *Luka*).
- **Luka:** Another instance of the student class. Luka is enrolled in the *Logics_and_AI* class which makes him a classmate of Liam when the reasoner is activated.
- **Mathieu:** Also an instance of the student class. Mathieu is enrolled in the *Data_Viz* class, he is classmate of Liam but doesn't share any classes with Luka. Therefore the reasoner does not infer the classmate relationship between these two entities.
- **France:** An instance representing the country.
- **Palaiseau, Paris:** Represent the cities of Palaiseau and Paris respectively in France. They are both instances of the *city* class and both are cities of France.
- **Telecom, Sorbonne:** Telecom and Sorbonne are instances of the *University* class. They have different locations (Palaiseau or Paris).
- **Computer Science, Philosophy, Maths:** These are instances of class *Major*, they are associated to programs in order to infer a student's major base on his program.

A screenshot showing these individuals defined in Protégé can be found in Figure 4.

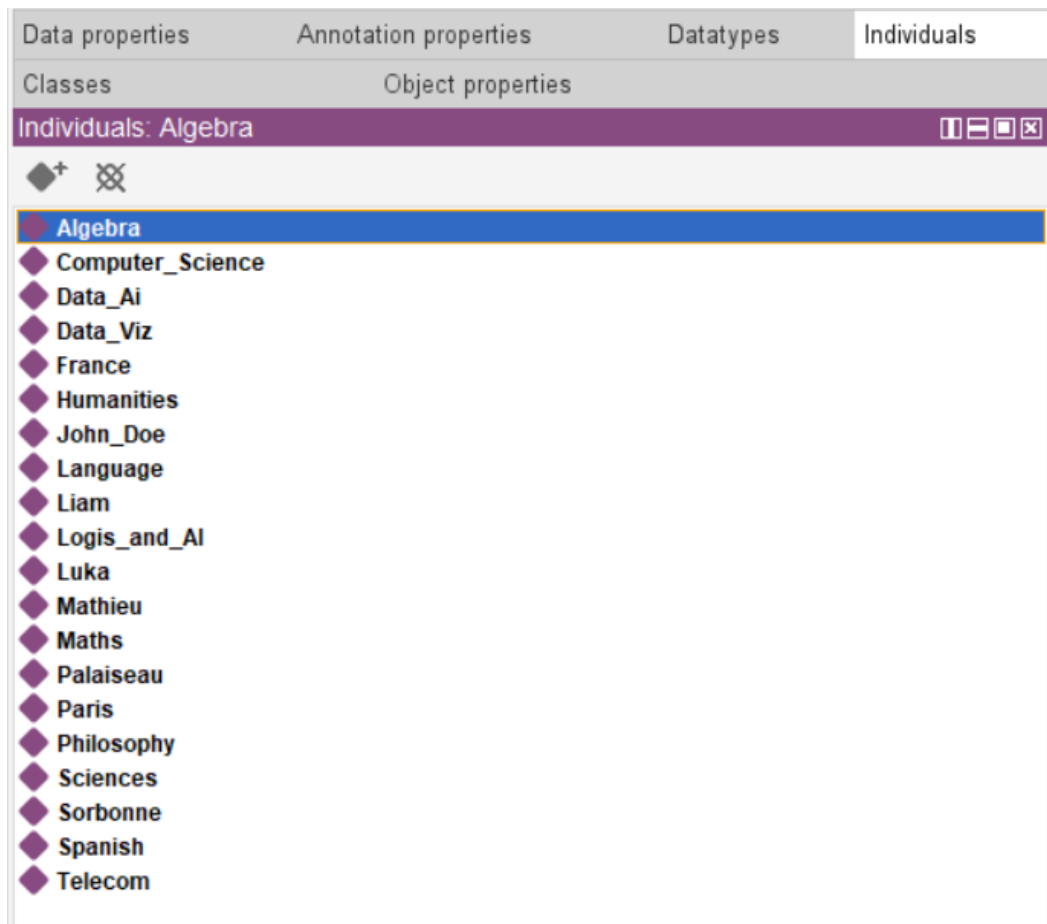


Figure 4: Snapshot of all the individuals in Protégé

3 Reasoning and Inferences

We used the *HermiT 1.4.3.456* reasoner for our ontology. Here are some examples of inferences we obtain:

1. In this example, shown in Figure 5, we can see that the reasoner infers a prerequisite. We defined the course *Data_Viz* having prerequisite *Logis*, and *Logis* having prerequisite *Algebra*. Therefore *Algebra* is an inferred prerequisite for *Data_Viz*.

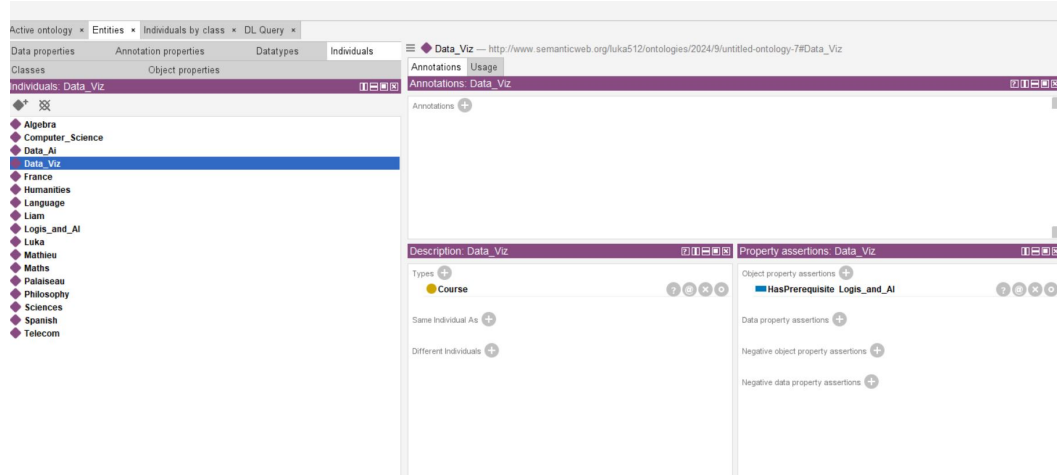


Figure 5: Before Starting the reasoner (example 1)

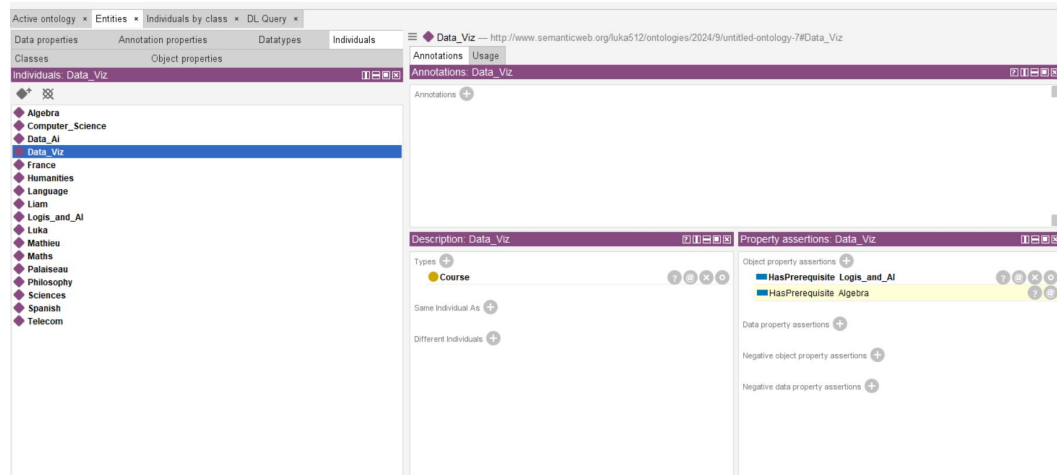


Figure 6: After Starting the reasoner (example 1)

2. In this example, shown in Figure 7, we can see how Liam's classmates are inferred. Liam and Luka are both enrolled in *Logis* and Liam and Mathieu are both enrolled in *Data Viz*. Therefore we have Mathieu and Luka inferred as classmates of Liam.

Furthermore we defined that Liam is attending the *Data AI* program and the *Data AI* program is associated to the *Computer Science* major. Using our ontology, the reasoner is able to find that Liam is majoring in *Computer Science*.

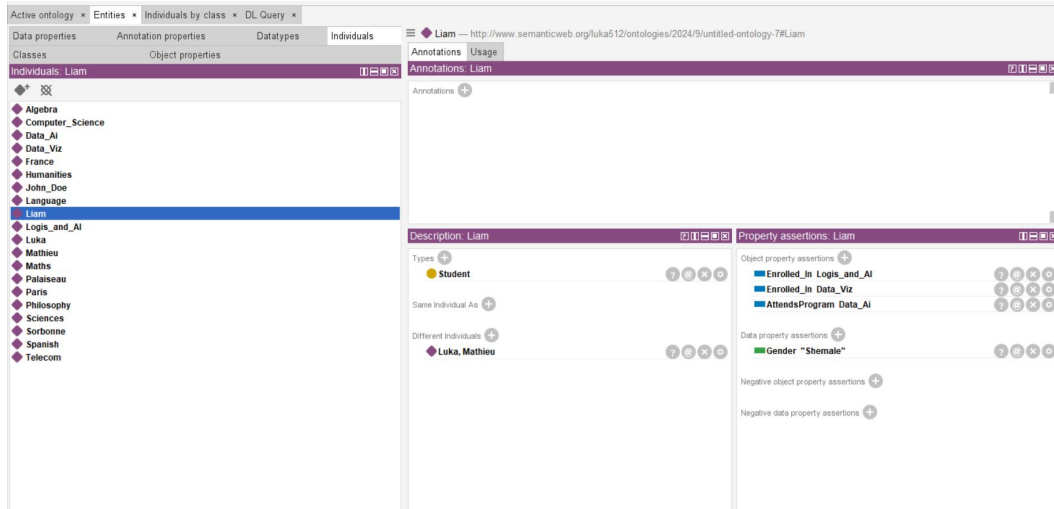


Figure 7: Before starting the reasoner (example 2)

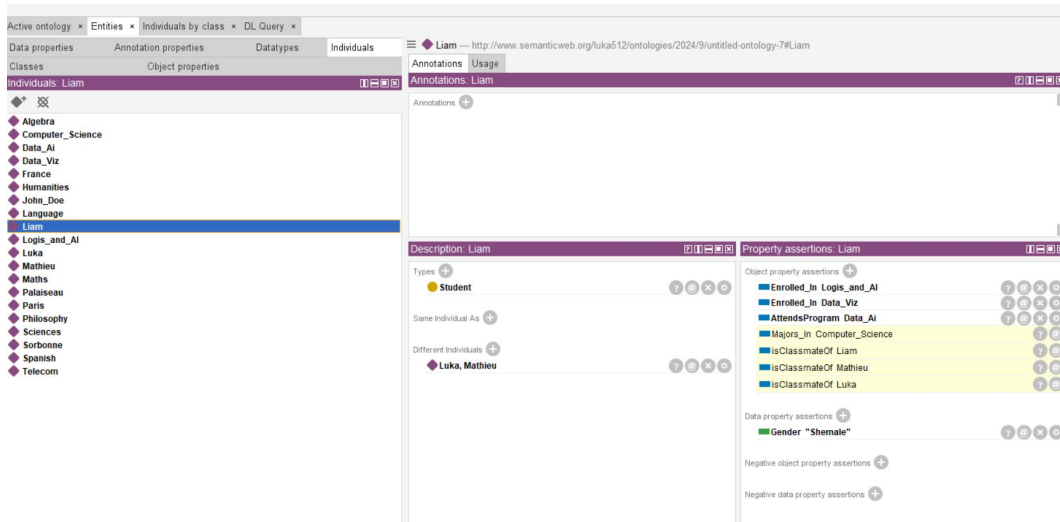


Figure 8: After starting the reasoner (example 2)

3. In this third example, as shown in Figure 8, the class of Languages is inferred. Given that the Spanish course belongs to Language, Language is inferred as a Major:

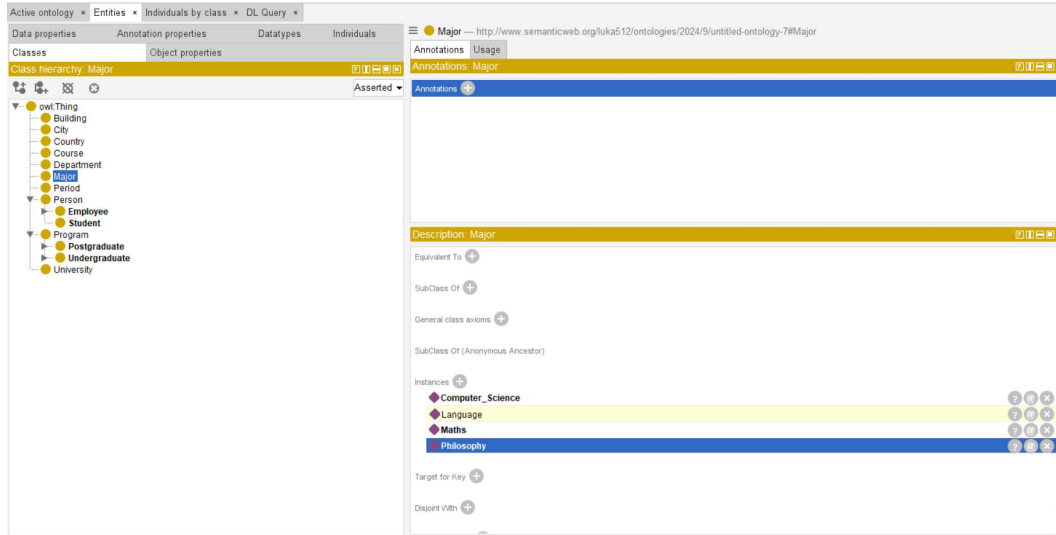


Figure 9: Inference of language as a major (example 3).

4 Example Queries

4.1 Queries Overview

To interact with our ontology and retrieve meaningful information, we used SPARQL queries. Querying enables us to extract specific data, explore relationships between entities, and validate the effectiveness of the ontology's structure.

4.2 Query Examples

1. List all students who are enrolled in a course taught by John Doe.

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>

SELECT ?student ?course WHERE {
  ?student rdf:type ont:Student .
  ?student ont:Enrolled_In ?course .
  { ?course ont:isTaughtBy ont:John_Doe }
  UNION
  { ont:John_Doe ont:Teaches ?course }
}
```

Result:

student	course
Mathieu	Data_Viz
Liam	Data_Viz

2. Find all courses that Liam is enrolled in.

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>

SELECT ?course WHERE {
  ont:Liam ont:Enrolled_In ?course .
}
```

}

Result:

course
Logis_and_AI Data_Viz

3. Find all classmates of Liam.

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?classmate WHERE {  
  ont:Liam ont:Enrolled_In ?course .  
  ?classmate ont:Enrolled_In ?course .  
  FILTER(?classmate != ont:Liam)  
}
```

Result:

classmate
Luka Mathieu

4. Find all courses that require Algebra as a prerequisite (directly or indirectly).

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?course WHERE {  
  ?course ont:HasPrerequisite+ ont:Algebra .  
}
```

Result:

course
Logis_and_AI Data_Viz

5. List all majors and the departments that offer them.

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?major ?department WHERE {  
  ?major rdf:type ont:Major .  
  ?major ont:isOfferedBy ?department .  
}
```

Result:

major	department
Computer_Science Philosophy	Sciences Humanities

6. Find all professors who teach courses in the Computer Science major.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?professor WHERE {  
  ?professor rdf:type ont:Professor .  
  ?professor ont:Teaches ?course .  
  ?course ont:Belongs_To ont:Computer_Science .  
}
```

Result:

professor
John_Doe

7. Find the majors that students are majoring in.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?student ?major WHERE {  
  ?student rdf:type ont:Student .  
  ?student ont:AttendsProgram ?program .  
  ?program ont:Associated_to ?major .  
}
```

Result:

student	major
Liam	Computer_Science

8. Find all universities located in France.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?university WHERE {  
  ?university ont:Located_In ?city .  
  ?city ont:isCityOf ont:France .  
}
```

Result:

university
Telecom Sorbonne

9. Find all courses that are part of the Computer Science major.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?course WHERE {  
  ?course ont:Belongs_To ont:Computer_Science .  
}
```

Result:

course
Data_Viz

10. Find the chain of prerequisites for Data_Viz.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?prerequisite WHERE {
  ont:Data_Viz ont:HasPrerequisite+ ?prerequisite .
}
```

Result:

prerequisite
Logis_and_AI
Algebra

11. List all professors along with the courses they teach.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?professor ?course WHERE {
  ?professor rdf:type ont:Professor .
  ?professor ont:Teaches ?course .
}
```

Result:

professor	course
John_Doe	Algebra
John_Doe	Data_Viz

12. Find all students and their genders.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?student ?gender WHERE {
  ?student rdf:type ont:Student .
  OPTIONAL { ?student ont:Gender ?gender . }
}
```

Result:

student	gender
Liam	Male
Luka	Male
Mathieu	Male

13. Find all employees and their roles.

```
PREFIX ont: <http://www.semanticweb.org/luka512/ontologies/2024/9/untitled-ontology-7#>
```

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
```

```
SELECT ?employee ?role WHERE {
  ?employee rdf:type ?role .
  ?role rdfs:subClassOf ont:Employee .
}
```

Result:

employee	role
John_Doe	Professor

14. List all departments and the universities they are part of.

```
PREFIX ont: <http://www.semanticweb.org/luca512/ontologies/2024/9/untitled-ontology-7#>
```

```
SELECT ?department ?university WHERE {
  ?department rdf:type ont:Department .
  ?department ont:PartOf ?university .
}
```

Result:

department	university
Humanities	Sorbonne
Sciences	Telecom
Language	Sorbonne

5 Conclusion

This report has detailed an illustrative practical example of an ontology describing a university system, including different entities such as classes for programs, departments, courses, and roles, as well as properties to define relationships between these elements.

To test and demonstrate the capabilities of our ontology, we created exemplary data by defining 20 individuals within the previously set up structure. The ontology was subsequently tested using Protégé's *HermiT* reasoner to infer further classes and additional relationships, such as indirect prerequisites for courses and inferred classmates.

Although the presented ontology already successfully captures the basic structure and relationships within a university, several enhancements could be considered. This includes expanding the ontology with even more data or making different elements more granular (e.g. tracking specific research interests, student grades, or other performance metrics). Additional queries could provide an even more comprehensive view of the ontology's capabilities.